

1. Pseudocode for Data Cleaning and Transformation on Time-Series Data

Objective

The algorithm should:

- Remove invalid entries
- Handle missing timestamps
- Interpolate missing values
- Detect and remove outliers
- Resample data at fixed intervals
- Maintain chronological consistency
- Use time indexing efficiently

Pseudocode:

ALGORITHM TimeSeries_Data_Cleaning_Transformation

INPUT:

Dataset D containing:

Timestamp

Value

Resampling interval R

Outlier threshold T

OUTPUT:

Cleaned and transformed time-series dataset C

BEGIN

STEP 1: LOAD DATA

Read dataset D

STEP 2: FILTER INVALID ENTRIES

Remove records where Timestamp is NULL

Remove records where Value is invalid or non-numeric

STEP 3: CONVERT TIMESTAMP

Convert Timestamp column into datetime format

STEP 4: REMOVE INVALID TIMESTAMPS

IF timestamp conversion fails THEN

Remove that record

END IF

STEP 5: REMOVE DUPLICATE TIMESTAMPS

Identify duplicate timestamps

IF duplicate timestamps exist THEN

Keep the first record

OR aggregate duplicate values using mean

END IF

STEP 6: SORT DATA CHRONOLOGICALLY

Sort D by Timestamp in ascending order

STEP 7: SET TIME INDEX

Set Timestamp as the index of D

STEP 8: IDENTIFY EXPECTED TIMESTAMPS

Generate a complete sequence of timestamps

from minimum timestamp to maximum timestamp

using interval R

STEP 9: INSERT MISSING TIMESTAMPS

Compare existing timestamps with expected timestamps

Add missing timestamps to the dataset

Assign missing values as NULL

STEP 10: INTERPOLATE MISSING VALUES

FOR each missing Value DO

Estimate value using neighboring timestamps

using linear interpolation

END FOR

STEP 11: DETECT OUTLIERS

Calculate mean μ of Value

Calculate standard deviation σ of Value

FOR each value x in D DO

Calculate:

$$Z = \text{ABS}((x - \mu) / \sigma)$$

IF $Z > T$ THEN

Mark x as an outlier

```

        END IF
    END FOR

STEP 12: HANDLE OUTLIERS

    Replace outlier values with NULL

    Interpolate missing values again

    OR remove outlier records

STEP 13: RESAMPLE TIME SERIES

    Resample data using fixed interval R

    IF multiple values occur within one interval THEN

        Calculate mean value

    END IF

STEP 14: VERIFY CHRONOLOGICAL CONSISTENCY

    Check whether timestamps are in ascending order

    IF timestamps are not sorted THEN

        Sort again

    END IF

STEP 15: FINAL VALIDATION

    Check for:

        Missing timestamps

        Missing values

        Duplicate timestamps

        Invalid values

        Outliers

STEP 16: STORE CLEAN DATA

    Save final dataset as C

RETURN C

END

```

Flow of the Algorithm

Raw Time-Series Data



Remove Invalid Records



Convert Timestamp to Date-Time



Sort Chronologically



Set Timestamp as Index



Detect Missing Timestamps



Insert Missing Time Intervals



Interpolate Missing Values



Detect Outliers



Remove / Replace Outliers



Resample at Fixed Intervals



Validate Data



Clean Time-Series Dataset

Efficiency Considerations

- Use the timestamp as an index for faster time-based operations.
- Sort the data only when required.
- Use vectorized operations for interpolation and outlier detection.
- Generate timestamps directly using the required fixed interval.
- Avoid unnecessary loops when processing large datasets.

2. Pseudocode for Feature Selection

Objective

The algorithm should:

- Normalize numerical features
- Measure correlation between features
- Remove irrelevant features
- Remove highly redundant features
- Select significant features
- Validate the selected features
- Reduce dimensionality while maintaining or improving model performance

Pseudocode:

ALGORITHM Feature_Selection

INPUT:

Dataset D

Target variable Y

Feature correlation threshold CT

Feature importance threshold IT

OUTPUT:

Selected feature set F_selected

BEGIN

STEP 1: LOAD DATA

Read dataset D

STEP 2: IDENTIFY FEATURES AND TARGET

Separate:

X = All input features

Y = Target variable

STEP 3: REMOVE INVALID FEATURES

FOR each feature F in X DO

IF F contains only missing values THEN

Remove F

END IF

IF variance of F = 0 THEN

Remove F

END IF

END FOR

STEP 4: HANDLE MISSING VALUES

FOR each numerical feature DO

Replace missing values with mean or median

END FOR

FOR each categorical feature DO

Replace missing values with mode

END FOR

STEP 5: NORMALIZE FEATURES

FOR each numerical feature X_i DO

Calculate:

$X_{\text{normalized}} =$

$(X_i - \text{minimum}(X_i))$

$/$

$(\text{maximum}(X_i) - \text{minimum}(X_i))$

Replace X_i with $X_{\text{normalized}}$

END FOR

STEP 6: CALCULATE FEATURE-TARGET CORRELATION

FOR each feature F_i DO

Calculate correlation:

$\text{Corr}_i = \text{Correlation}(F_i, Y)$

END FOR

STEP 7: REMOVE IRRELEVANT FEATURES

FOR each feature F_i DO

IF $\text{ABS}(\text{Corr}_i) < \text{IT}$ THEN

Remove F_i

END IF

END FOR

STEP 8: CALCULATE FEATURE-FEATURE CORRELATION

Create correlation matrix M

containing correlation between every pair

of remaining features

STEP 9: REMOVE REDUNDANT FEATURES

FOR each pair of features

(F_i, F_j) DO

IF $\text{ABS}(\text{Correlation}(F_i, F_j)) > \text{CT}$ THEN

Compare:

$\text{ABS}(\text{Correlation}(F_i, Y))$

and

$\text{ABS}(\text{Correlation}(F_j, Y))$

Remove the feature having lower

correlation with the target

END IF

END FOR

STEP 10: CREATE SELECTED FEATURE SET

F_selected =

All remaining significant features

STEP 11: TRAIN VALIDATION MODEL

Split dataset into:

Training Data

Testing Data

Train Model_1 using all features

Calculate:

Performance_All

Train Model_2 using F_selected

Calculate:

Performance_Selected

STEP 12: VALIDATE FEATURE SELECTION

Calculate:

Number_of_Features_Removed

Compare:

Performance_Selected

with

Performance_All

IF Performance_Selected >= Performance_All

OR performance reduction is acceptable THEN

Accept F_selected

ELSE

Adjust thresholds CT and IT

Repeat feature selection process

END IF

STEP 13: RETURN RESULT

Return F_selected

END

Feature Selection Process

Original Dataset



Separate Features and Target



Remove Missing / Invalid Features



Remove Constant Features



Normalize Numerical Features



Calculate Feature-Target Correlation



Remove Irrelevant Features



Calculate Feature-Feature Correlation



Remove Highly Correlated Features



Select Significant Features



Train Model Using Selected Features



Compare Performance



Final Reduced Feature Dataset